

VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMATIKOS INSTITUTAS
INFORMATIKOS KATEDRA

Labaratorinis darbas

Optimizavimas be apribojimų

Atliko: 4 kurso 3 grupės studentas

Naglis Kontautas (parašas)

Darbo vadovas:

a (parašas)

Vilnius
2024

Turinys

1. Įvadas	2
1.1. Užduotis	2
1.2. Metodai ir formulės	2
1.2.1. Kintamieji	2
1.2.2. Tikslų funkcija.....	3
1.2.3. Dalinės išvestinės	3
1.3. Kita reikalinga informacija	3
2. Gradientinis nusileidimas	4
2.1. Gradientinio nusileidimo rezultatai	6
2.2. Gradientinis nusileidimas. Pradinis taškas (0; 0).....	6
2.3. Gradientinis nusileidimas. Pradinis taškas (1; 1).....	6
2.4. Gradientinis nusileidimas. Pradinis taškas (0,8; 0).....	7
3. Greičiausias nusileidimas	10
3.1. Greičiausio nusileidimo rezultatai.....	12
3.2. Greičiausias nusileidimas. Pradinis taškas (0; 0)	12
3.3. Greičiausias nusileidimas. Pradinis taškas (1; 1)	12
3.4. Greičiausias nusileidimas. Pradinis taškas 0,8; 0).....	14
4. Deformuotas simpleksas	16
4.1. Deformuoto simplekso rezultatai	21
5. Deformuotas simpleksas. Pradinis taškas 0; 0)	22
6. Deformuotas simpleksas. Pradinis taškas 1; 1)	25
7. Deformuotas simpleksas. Pradinis taškas (0,8; 0)	29
8. Visų metodų rezultatai	30
9. Išvados	31

1. Įvadas

Darbo tikslas yra suprogramuoti gradientinio nusileidimo, greičiausiojo nusileidimo ir deformuojamo simplekso algoritmus ir palyginti jų rezultatus bei veikimo principus.

1.1. Užduotis

Laikant kintamaisiais dėžės priekinės ir galinės sienų plotų sumą, šoninių sienų plotų sumą, viršutinės ir apatinės sienų plotų sumą, aprašyti vienetinio dėžės paviršiaus ploto reikalavimą ir dėžės tūrio pakelto kvadratu funkciją.

1.2. Metodai ir formulės

Vienetinės dėžės ploto formulė:

$$2ab + 2ac + 2bc = 1 \quad (1)$$

a, b, c yra dėžės briaunos.

1.2.1. Kintamieji

Briaunas galime išreikšti per kintamuosius:

$$x = 2ab, y = 2ac, z = 2bc \quad (2)$$

Dėžės ploto formulė:

$$x + y + z = 1 \quad (3)$$

Vis dar turime 3 skirtingi kintamieji, todėl galime vieną kintamąjį išsireikšti per kitus:

$$z = 1 - x - y \quad (4)$$

Dėžės tūrio formulė:

$$V = a \cdot b \cdot c \quad (5)$$

Kadangi matmenys negali būti neigiami, sudaromas kvadratinio tūrio formulė:

$$V^2 = a^2 * b^2 * c^2 \quad (6)$$

$$x \cdot y \cdot z = 2ab \cdot 2ac \cdot 2bc = 2^3 \cdot (a^2 b^2 c^2) = 8 \cdot V^2 = x \cdot y \div 8 \cdot (1 - x - y) \quad (7)$$

1.2.2. Tikslo funkcija

$$V^2 = x \cdot y \div 8 \cdot (1 - x - y) \quad (8)$$

Kadangi reikia rasti funkcijos minimumą tai funkcija apverčiama: $\min(-V^2)$. Pavadinkime ją funkcija $G(x, y)$:

$$G(x, y) = -x \cdot y \div 8 \cdot (1 - x - y) \quad (9)$$

1.2.3. Dalinės išvestinės

$$G'_x = y \cdot (2x + y - 1) \div 8, \quad (10)$$

$$G'_y = x \cdot (2y + x - 1) \div 8 \quad (11)$$

1.3. Kita reikalinga informacija

Pradiniai taškai: $X_0 = (0; 0)$, $X_1 = (1; 1)$, $X_m = (0,8; 0)$.

Pasirinkta slenkstinė reikšmė: $\epsilon = 10^{-5}$.

Pagal tikslo funkciją [1.2.2], minimumo taškas yra $(\frac{1}{3}; \frac{1}{3})$.

2. Gradientinis nusileidimas

Gradientinio nusileidimo veikimas yra pagrįstas tiklos funkcijos išvestine, šiuo atveju dalinėmis funkcijos išvestinėmis [1.2.3], bei iteracijomis. Išvestinių pagalba su kiekviena iteracija galime judėti funkcijos ekstremumo taško link. Kadangi šiai užduočiai naudojamos koordinatės x ir y , koordinačių reikšmės yra keičiamos pagal jų atitinkamas išvestines (dalinius gradientus). Judama antigradiento linkme (nes ieškome minimumo [1.2.2]), todėl:

$$x_{new} = x_{old} - G'_x \quad (12)$$

$$y_{new} = y_{old} - G'_y \quad (13)$$

Kadangi, su kiekviena iteracija gaunamos kitokios x ir y reikšmės, todėl kiekvienoje iteracijoje iš naujo yra apskaičiuojami ir gradientai G'_x ir G'_y . Tokiu būdu kiekvienoje iteracijoje ir koordinačių, ir gradientų reikšmės mažėja (nes einame link minimumo). Ciklas užbaigiamas, kai gradientų reikšmė yra pakankamai maža, pavyzdžiui, mažiau už ϵ . Pasibaigus ciklui, turimos koordinačių reikšmės bus tokios, su kuriomis tikslo funkcija įgauna reikšmę labai artimą mažiausiai funkcijos reikšmei.

```
1 private (List<(double, double, double)>, int, int) gradientDescent(double x, double y,
2     double epsilon)
3 {
4     List<(double, double, double)> pointsList = new ();
5     int i = 0;
6     int targetFunctionCalled = 0;
7
8     while (i < 10000)
9     {
10         (double gradX, double gradY) = PartialGradients(x, y);
11         targetFunctionCalled += 2;
12         if (Math.Abs(gradX) < epsilon && Math.Abs(gradY) < epsilon)
13             break;
14
15         x -= gradX;
16         y -= gradY;
17         pointsList.Add((x, y, 1));
18         i++;
19     }
20     return (pointsList, targetFunctionCalled, i);
21 }
```

1 pav. Gradientinio nusileidimo kodas

8 - 19 eil. vyksta ciklas, kuris baigiasi jei bent vienas gradientas yra mažesnis už ϵ .

10 eil. apskaičiuojami gradientai pagal x ir y koordinates.

12 eil. tikrinama ar reikia nutraukti ciklą.

15 - 16 eil. keičiamos koordinas pagal antigradientą.

17 eil. išsaugomi žingsniai.

20 eil. grąžinami žingsniai, funkcijos kvietimų skaičius ir metodo prasisukusių ciklų kiekis.

2.1. Gradientinio nusileidimo rezultatai

2.2. Gradientinis nusileidimas. Pradinis taškas (0; 0)

Su pradiniu tašku (0; 0) gradientinis bei greičiausi nusileidimai neveikia, kadangi tai yra tokie kintamieji, su kuriais tikslo funkcija lygi nuliui. Gradientai taip pat nuliai, todėl koordinatės nekinta. $G(0,0) = 0$, $G'_x(0) = 0$, $G'_y(0) = 0$.

2.3. Gradientinis nusileidimas. Pradinis taškas (1; 1)

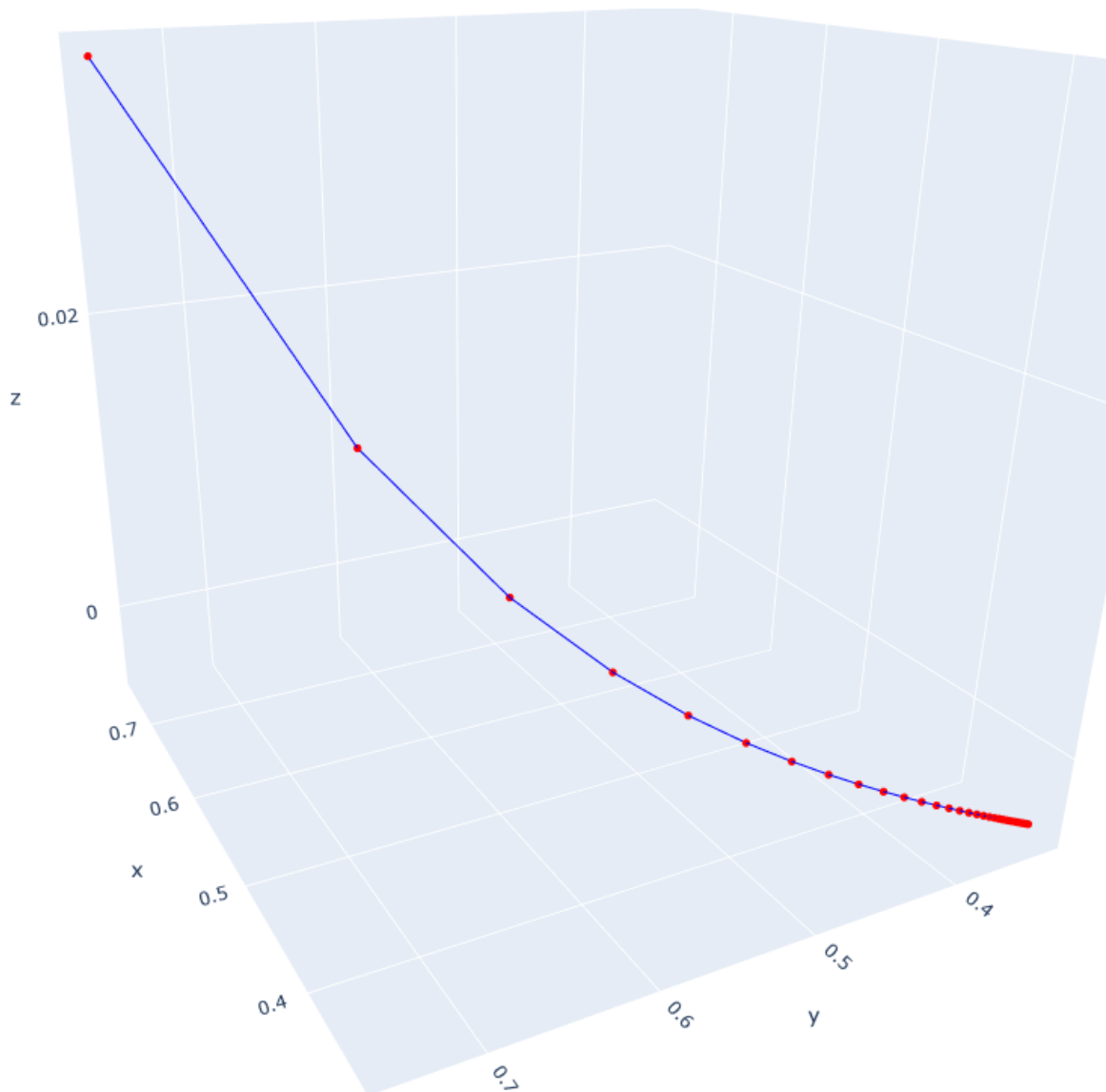
Tikslo funkcija kviesta 120 kartu, per 59 ciklus.

Metodas pasibaigė, nes gradientų reikšmės mažesnės už epsilon 1E-05: gradX: 8.879519396919052E-06, gradY: 8.879519396919052E-06

2 pav. Gradientinio nusileidimo ciklo pabaigos rodiklis

1 lentelė. Gradientinio nusileidimo tarpinės koordinatės su (1; 1) pradiniu tašku

x	y
1	1
0.75	0.75
0.6328125	0.6328125
0.5617446899414062	0.5617446899414062
0.5136288649300695	0.5136288649300695
0.4789019939628214	0.4789019939628214
0.45275957327508676	0.45275957327508676
.	.
.	.
.	.
0.33343935994333324	0.33343935994333324
0.3334261024014675	0.3334261024014675
0.3334145030406632	0.3334145030406632
0.33340435435655147	0.33340435435655147



3 pav. Gradientinio nusileidimo judėjimas iš (1; 1) pradinio taško

2.4. Gradientinis nusileidimas. Pradinis taškas (0.8; 0)

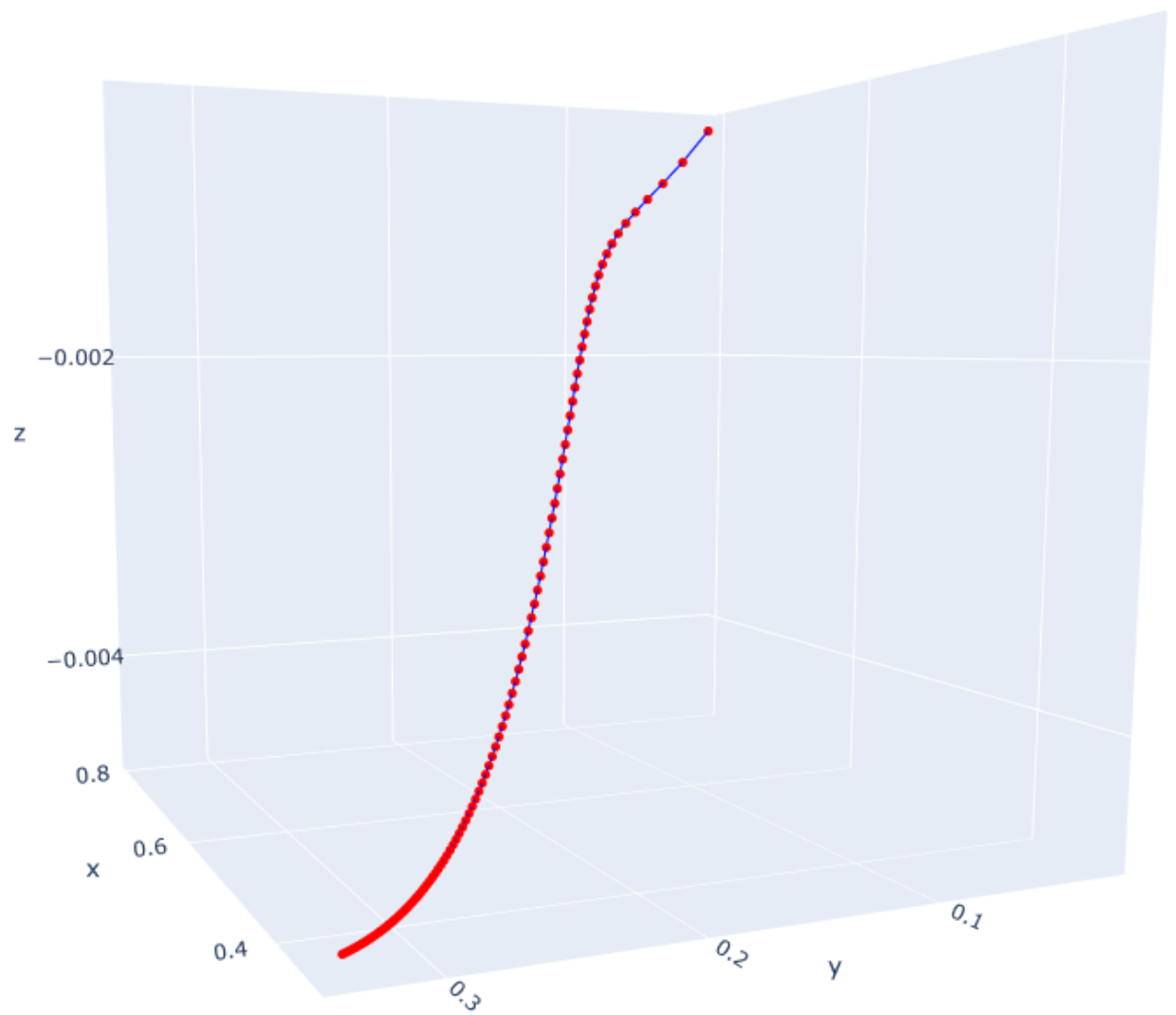
Funkcija kviesta 392 kartų; Ciklu kiekis: 195

Metodas pasibaigė, nes gradientu reikšmės mažesnės už epsilon 1E-05: gradX: 9.810346915486388E-06, gradY: -9.784395925811435E-06

4 pav. Gradientinio nusileidimo ciklo pabaigos rodiklis

2 lentelė. Gradientinio nusileidimo tarpinės koordinatės su (0.8; 0) pradiniu tašku

x	y
0.8	0
0.8	0.01999999999999997
0.79845	0.03599999999999999
0.79560195	0.04892989968749999
0.791686739674339	0.059525154680854835
0.7869031595949992	0.06835869150864995
0.7814159646119995	0.07587159605196662
.	.
.	.
.	.
0.3336123325175429	0.33305478131392263
0.33360069864971065	0.33306637871507655
0.33358955025127873	0.33307749361982386
0.3335788670353837	0.3330881460727623
0.33356862956523814	0.3330983552881419



5 pav. Gradientinio nusileidmio judėjimas iš $(0.8; 0)$ pradinio taško

3. Greičiausias nusileidimas

Greičiausias nusileidimas pagrįstas gradientinio nusileidimo metodu [2]. Tačiau, šiame metode gradientų reikšmės turi žingsnio ilgį kaip daugiklį λ . Šis kintamasis koreguoja kaip greitai ar lėtai su antigradientu judama link funkcijos minimumo:

$$x_{new} = x_{old} - \lambda \cdot G'_x \quad (14)$$

$$y_{new} = y_{old} - \lambda \cdot G'_y \quad (15)$$

λ yra parenkamas naudojant vienmačio optimizavimo metodą - auksinio pjūvio metodą. Pasirenkamas intervalas $[0; 5]$, kurį auksinio pjūvio metodas iteracijomis siaurina link funkcijos reikšmės minimumo. Pagal parametą $\sigma = (-1 + \sqrt{5}) \div 2$ intervale pažymimi du taškai λ_1 ir λ_2 . Tikrinamos funkcijos reikšmės: $G((x - \lambda_k \cdot G'_x), (y - \lambda_k \cdot G'_y))$, $k = \{1, 2\}$. Kadangi einam link minimumo, todėl didesnės reikšmės pusė yra atmetama ir susiaurinamas intervalas. Kitoje iteracijoje vykdomi tokie patys veiksmai, tik jau su vis siaurinamu intervalu. Siaurinimas, taip pat ir metodas, užbaigiamas, kai gradinetų reikšmė yra pakankamai maža, pavyzdžiui, mažiau už ϵ . Pasibaigus ciklui, turimos koordinatų reikšmės bus tokios, su kuriomis tikslo funkcija įgauna reikšmę labai artimą mažiausiai funkcijos reikšmei.

```
1      private (List<(double, double, double)>, int, int) FastestDescent(double x, double y,
2          double epsilon)
3  {
4      List<(double, double, double)> pointTimeLine = new();
5      pointTimeLine.Add((x, y, 1));
6      int cycles = 0;
7      int targetFunctionCalled = 0;
8      double t = (-1 + Math.Sqrt(5)) / 2;
9      double leftBorder = 0;
10     double rightBorder = 5;
11
12     while (cycles < 5000)
13     {
14         (double gradX, double gradY) = PartialGradients(x, y);
15         targetFunctionCalled += 2;
16         if (Math.Abs(gradX) < epsilon && Math.Abs(gradY) < epsilon)
17         {
18             break;
19         }
20
21         double interval = rightBorder - leftBorder;
22         double lambda1 = rightBorder - t * interval;
23         double lambda2 = leftBorder + t * interval;
24
25         double f1 = ObjectiveFunction(x - lambda1 * gradX, y - lambda1 * gradY);
```

```

26     double f2 = ObjectiveFunction(x - lambda2 * gradX, y - lambda2 * gradY);
27     targetFunctionCalled += 2;
28
29     while (interval > epsilon)
30     {
31         if (f2 < f1)
32         {
33             leftBorder = lambda1;
34             lambda1 = lambda2;
35             f1 = f2;
36             interval = rightBorder - leftBorder;
37             lambda2 = leftBorder + t * interval;
38             f2 = ObjectiveFunction(x - lambda2 * gradX, y - lambda2 * gradY);
39             targetFunctionCalled++;
40         }
41         else if (f1 < f2)
42         {
43             rightBorder = lambda2;
44             lambda2 = lambda1;
45             f2 = f1;
46             interval = rightBorder - leftBorder;
47             lambda1 = rightBorder - t * interval;
48             f1 = ObjectiveFunction(x - lambda1 * gradX, y - lambda1 * gradY);
49             targetFunctionCalled++;
50         }
51         else
52         {
53             leftBorder = lambda1;
54             rightBorder = lambda2;
55             interval = rightBorder - leftBorder;
56             lambda1 = rightBorder - t * interval;
57             lambda2 = leftBorder + t * interval;
58             f1 = ObjectiveFunction(x - lambda1 * gradX, y - lambda1 * gradY);
59             f2 = ObjectiveFunction(x - lambda2 * gradX, y - lambda2 * gradY);
60             targetFunctionCalled += 2;
61         }
62         cycles++;
63     }
64     double lambda = (f1 < f2) ? lambda1 : lambda2;
65     x -= lambda * gradX;
66     y -= lambda * gradY;
67     pointTimeLine.Add((x, y, lambda));
68     cycles++;
69 }
70
71 return (pointTimeLine, targetFunctionCalled, cycles);
72 }

```

6 pav. Greičiausio nusileidimo su auksiniu pjūviu kodas

7 eil. gaunama σ reikšmė.

8 - 9 eil. nustatomos pradinės intervalo ribos.

11 eil. prasideda gradientinio nusileidimo (greičiausio nusileidimo) ciklas.

13 eil. apskaičiuojami gradientai pagal x ir y koordinates.

15 - 20 eil. tikrinama ar reikia nutraukti ciklą.

22 - 24 eil. gaunamos λ_1 ir λ_2 .

26 - 27 eil. apskaičiuojamos funkcijos reikšmės f_1 pagal λ_1 ir f_2 pagal λ_2 .

30 eil. prasideda auksinio pjūvio metodo ciklas

32 - 51 eil. lyginamos funkcijų, su skirtingais žingsniais, reikšmės. Jeigu f_1 yra didesnė reikšmė, intervalas siaurinamas is kairės pusės, jei f_2 didesnė reikšmė, intervalas siaurinamas is dešinės pusės. Apskaičiuomamos naujos funkcijos reikšmės kitų iteracijų lyginimams ir intervalo siaurinimams.

52 eil. jei $f_1 = f_2$, tai siaurinamas intervalas iš abiejų pusių. (Šis variantas beveik niekada nenu-tinka)

63 eil. nors abiejų λ_1 ir λ_2 reikšmės labai panašios, paimama mažesnė.

66 - 67 eil. keičiamos koordinačių reikšmės pagal anti gradientą ir λ .

68 eil. išsaugomi tarpiniai judėjimo taškai.

72 eil. gražinami žingsniai, funkcijos kvietimų skaičius ir metodo prasisukusių ciklų kiekis.

3.1. Greičiausio nusileidimo rezultatai

3.2. Greičiausias nusileidimas. Pradinis taškas (0; 0)

Su pradiniu tašku $(0; 0)$ gradientinis bei greičiausi nusileidimai neveikia, kadangi tai yra tokie kin-tamieji, su kuriais tikslo funkcija lygi nuliui. Gradientai taip pat nuliai, todėl koordinates nekinta. $G(0,0) = 0$, $G'_x(0) = 0$, $G'_y(0)$.

3.3. Greičiausias nusileidimas. Pradinis taškas (1; 1)

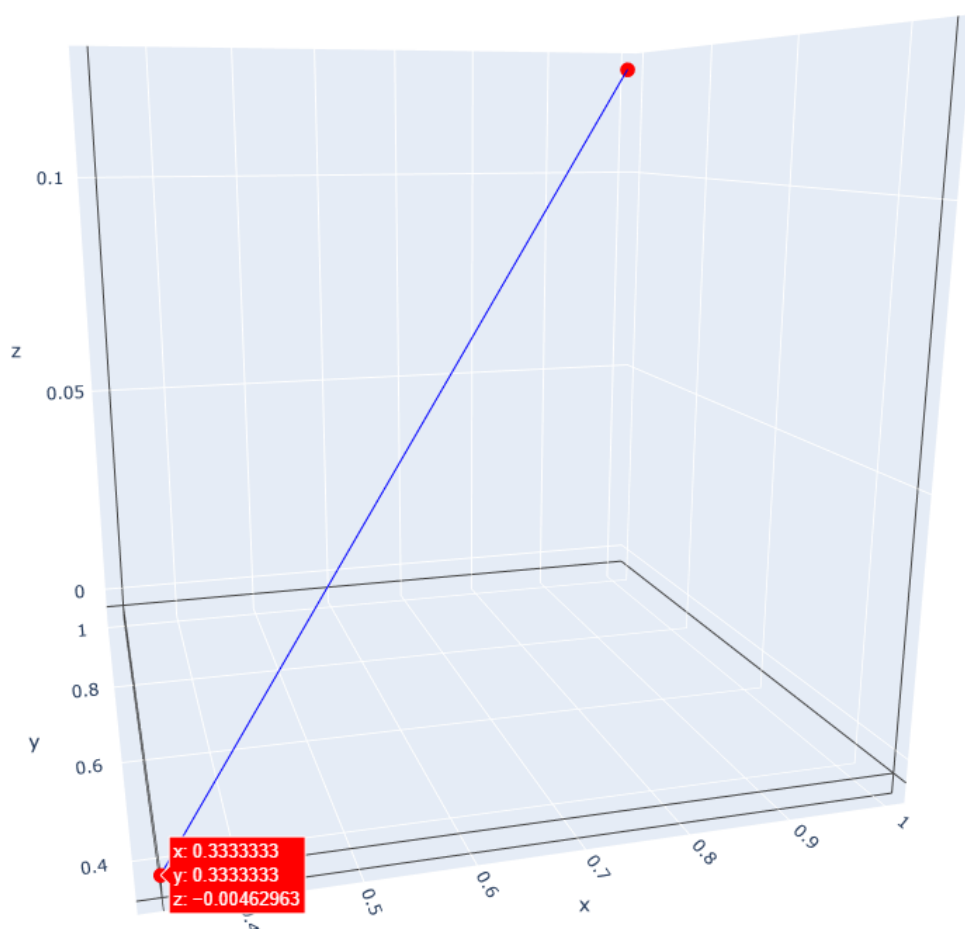
Funkcija kviesta 34 kartų; Ciklu kiekis: 29

Metodas pasibaigė, nes gradientu reikšmės mažesnės už epsilon 1E-05: gradX: -6.103707306777281E-09, gradY: -6.103707306777281E-09

7 pav. Greičiausio nusileidimo ciklo pabaigos rodiklis

3 lentelė. Greičiausio nusileidimo tarpinės koordinatės su (1; 1) pradiniu tašku

x	y	λ
1	1	1
0.3333332845036677	0.3333332845036677	2.666666861985329



8 pav. Greičiausio nusileidimo judėjimas iš (1; 1) pradinio taško

3.4. Greičiausias nusileidimas. Pradinis taškas 0,8; 0)

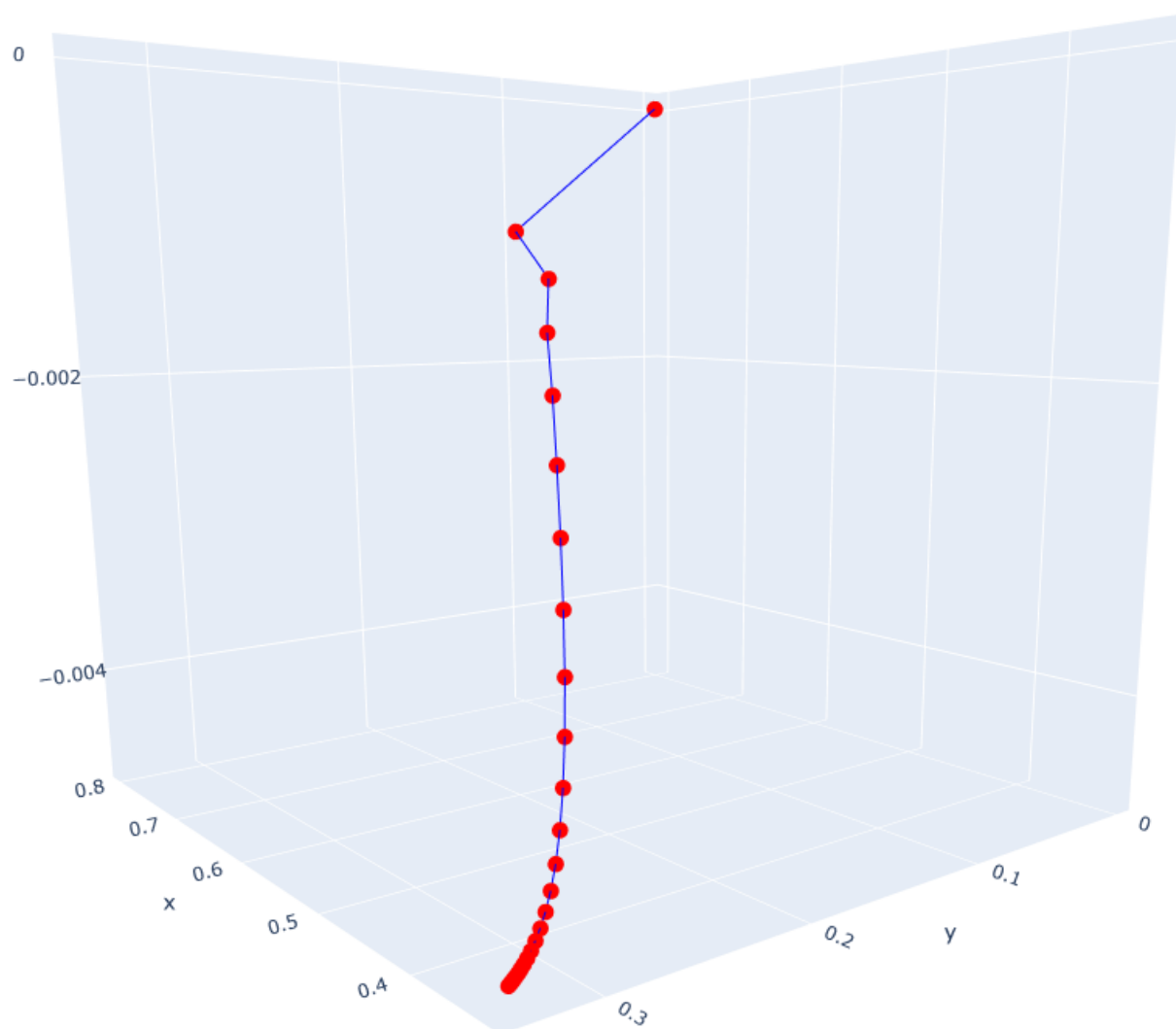
Tikslo funkcija kviesta 178 kartų, per 65 ciklus.

Metodas pasibaigė, nes gradientu reikšmės mažesnės už epsilon 1E-05: gradX: 8.440557100035939E-06, gradY: -8.42537319655692E-06

9 pav. Greičiausio nusileidimo ciklo pabaigos rodiklis

4 lentelė. Greičiausio nusileidimo tarpinės koordinatės su (0,8; 0) pradiniu tašku

x	y	λ
0.8	0	1
0.8	0.099999994625094999	4.9999973125475
0.7562500503897182	0.099999999999997108	4.9999973125475
0.7179688146668278	0.12067867738372687	4.9999973125475
0.6759865124163857	0.13893025627318897	4.9999973125475
.	.	.
.	.	.
.	.	.
0.33384924609756755	0.332818738729843	4.9999973125475
0.33374165606534417	0.3329258367681854	4.9999973125475
0.3336565209614033	0.3330106635052145	4.9999973125475
0.33358914764801234	0.3330778435471928	4.9999973125475
0.3335358263182049	0.3331310437440937	4.9999973125475



10 pav. Greičiausio nusileidimo judėjimas iš $(0.8; 0)$ pradinio taško

4. Deformuotas simpleksas

Deformuotam simpleksui buvo pasirinktas Nelder-Mead metodas. Deformuotas dėl to, kad nėra palaikoma tiklingojo daugiakampio (šiuo atveju trikampio) forma, tačiau daugiakampis visada išliks bent jau lygia šonis. Nelder-Mead simplekso algoritmo principas yra pagrįstas trejų viršūnių atspindžiais pagal priešais esančios briaunos vidurio taško padėtį. Metodas veikia iteracijomis, o per vieną iteraciją įvykdomas vieno taško atspindys.

Pagal pasirinktą pradinį tašką sukuriamas simpleksas ir apskaičiuojama tikslo funkcijos reikšmė visuose trikampio viršūnėse. Kiekvienos iteracijos pradžioje yra surikiuojami taškai pagal funkcijos reikšmes, kad galėtume taškus suskirstyti pagal tinkamumą. Kadangi mums reikia ieškoti funkcijos minimumo taško, vadinasi geriausias taško tinkamumas yra tame taške mažiausia funkcijos reikšmė.

X_w . blogiausias taškas - funkcijos reikšmė šiam taške yra didžiausia iš simplekso taškų.

X_v . vidurinis taškas - funkcijos reikšmė šiam taške yra nei didžiausia, nei mažiausia iš simplekso taškų.

X_b . geriausias taškas - funkcijos reikšmė šiam taške yra mažiausia iš simplekso taškų.

X_r . atspindėtas taškas, kitaip - naujasis taškas.

X_c . priešais bandomo atspindėti taško esančios briaunos vidurinis taškas.

ϕ . parametras, kuris nurodo X_r taško poziciją pagal atspindėti bandomą tašką.

Daromas blogiausio taško (taško, kuriame funkcijos reikšmė yra didžiausia iš dabartinių simplekso taškų) atspindys su $\phi = 1$:

$$X_r = X_w + (1 + \phi) \cdot (X_c - X_w) \quad (16)$$

Atspindėtas taškas visada pakeičia simplekso blogiausią tašką - taip išlaikomas trijų taškų matmuo, bei simplexas mažėja link minimumo taško. Lyginama atspindėtame taške funkcijos reikšmė su dabartinio simpleksu ($f(X_*)$ - yra funkcijos reikšmė nuo X_*):

1. jei $f(X_b) < f(X_r) < f(X_v)$, tai atspindėtas taškas yra geroje vietoje ir kiti veiksmi neatliekami.
2. jei $f(X_r) < f(X_b)$, tai daromas simplekso išplėtimas, o $\phi = 2$. Taip daroma todėl, kad X_r yra geresnis nei geriausias simplekso taškas - vadinasi jo atspindys eina palankia kryptimi. Dėl to, bandoma žengti dar toliau. Jeigu $f(X_{r2}) > f(X_r)$, paliekamas X_r kaip naujas simplekso taškas, jei $f(X_{r2}) < f(X_r)$, vadinasi simplexas praplėčiamas.
3. jei $f(X_r) > f(X_w)$, tai žiūrima ar atspindys su $\phi = -0,5$ yra geresnis už X_v . Jei taip - simplexas suspaudžiamas.

4. jei $f(X_v) < f(X_r) < f(X_w)$, tai žiūrima ar atspindys su $\phi = 0,5$ yra geresnis už X_v . Jei taip - simplexas susiaurinamas.
5. jei **3** ir **4** veiksmuose X_r yra prastesnis už X_v , visas simpleksas yra mažinamas. Mažinamo procesas: pagal geriausią turimą simplekso tašką yra sutrumpinamos trikampio briaunos per pusę. Tada apskaičiuojama funkcijos reikšmė naujuose dviejuose taškuose. Mažinimas vykdomas dėl to, kad jei atspindžio taškas su $\phi = \pm 0,5$ yra blogesnis už X_v , tai jį pridėjus prie simplekso, jis vis tiek bus blogiausias taškas kitoje iteracijoje.

Iteracijos pasibaigia, kai simplekso ilgiausios kraštinės ilgis yra mažesnis už $\epsilon = 10^{-5}$.

```

1 namespace OptimizavimasBeApribojimu
2 {
3     public class Simplex
4     {
5         public Simplex() { }
6
7         public (List<double[] []>, int, int) Run(double x, double y, double epsilon)
8         {
9             List<double[] []> simplexTimeLine = new();
10            double atspindys = 1;
11            double sumazintasAtspindys = 0.5;
12            double istemptasAtspindys = 2;
13            double edgeSize = 1;
14
15            double[] [] simplex = GenerateSimplex(x, y, edgeSize);
16            int functionCallsCount = 3;
17            int cycles = 0;
18
19            while (getLongestEdgeLength(simplex) > epsilon)
20            {
21                Array.Sort(simplex, (a, b) => b[2].CompareTo(a[2]));
22                simplexTimeLine.Add((double[] [])simplex.Clone());
23                Console.WriteLine($"F-jos reikšmės: {simplex[0][2]}, {simplex[1][2]},
24                                {simplex[2][2]}");
25                Console.WriteLine($"Simplekso taškai: ({simplex[0][0]}; {simplex[0][1]})
26                                | ({simplex[1][0]}; {simplex[1][1]}) | ({simplex[2][0]}; {simplex[2][1]}) |");
27                cycles++;
28
29                double[] best = simplex[2];
30                double[] worst = simplex[0];
31                double[] secondWorst = simplex[1];
32
33                double[] centroid = CalculateCentroid(simplex, worst);
34                double[] newPoint = Reflect(centroid, worst, atspindys);
35
36                (simplex, int count, bool isNegative) = FixNegative(simplex, newPoint,

```

```

37         centroid, -sumazintasAtspindys);
38     functionCallsCount += count;
39     if (isNegative)
40         continue;
41     newPoint[2] = ObjectiveFunction(newPoint[0], newPoint[1]);
42     functionCallsCount++;
43
44     Console.WriteLine($"Atsipindys: {newPoint[2]}");
45
46     if (newPoint[2] > best[2] && newPoint[2] < secondWorst[2])
47     {
48         simplex[0] = newPoint;
49         Console.WriteLine($"liko toks pats: {newPoint[2]}\n");
50         continue;
51     }
52     else if (newPoint[2] < best[2])
53     {
54         double[] extendedPoint = Reflect(centroid, worst, istemptasAtspindys);
55         extendedPoint[2] = ObjectiveFunction(extendedPoint[0], extendedPoint[1]);
56         functionCallsCount++;
57         (simplex, count, isNegative) = FixNegative(simplex, newPoint, centroid,
58             -sumazintasAtspindys);
59         functionCallsCount += count;
60         if (isNegative)
61             continue;
62         simplex[0] = extendedPoint[2] < newPoint[2] ? extendedPoint : newPoint;
63         Console.WriteLine($"Istemptas: {simplex[0][2]}\n");
64     }
65     else
66     {
67         double[] compressedPoint = Reflect(centroid, worst, sumazintasAtspindys);
68         compressedPoint[2] = ObjectiveFunction(compressedPoint[0], compressedPoint[1]);
69         functionCallsCount++;
70         if (compressedPoint[2] < secondWorst[2])
71         {
72             simplex[0] = compressedPoint;
73             Console.WriteLine($"Suspaustas: {simplex[0][2]}\n");
74         }
75         else
76         {
77             (simplex, int calls) = ContractSimplex(simplex, centroid,
78                 sumazintasAtspindys);
79             functionCallsCount += calls;
80         }
81     }
82 }
83 return (simplexTimeLine, functionCallsCount, cycles);

```

```

84     }
85
86     double ObjectiveFunction(double x, double y)
87     {
88         return -x * y * (1 - x - y) / 8;
89     }
90
91     private (double[][], int) ContractSimplex(double[][] simplex, double[] centroid,
92         double phi)
93     {
94         double[] contractedPoint = Reflect(centroid, simplex[0], -phi);
95         contractedPoint[2] = ObjectiveFunction(contractedPoint[0], contractedPoint[1]);
96         int functionCallsCount = 1;
97         if (contractedPoint[2] < simplex[1][2])
98         {
99             simplex[0] = contractedPoint;
100             Console.WriteLine($"Suspaustas: {simplex[0][2]}\n");
101         }
102         else
103         {
104             Console.WriteLine("Simpleksas sumazejo\n");
105             simplex = ShrinkSimplex(simplex);
106             functionCallsCount += 2;
107         }
108         return (simplex, functionCallsCount);
109     }
110
111     private (double[][], int, bool) FixNegative(double[][] simplex, double[] point,
112         double[] centroid, double phi)
113     {
114         int functionCallsCount = 0;
115         bool isNegative = false;
116         if (point[0] < 0 || point[1] < 0)
117         {
118             (simplex, int calls) = ContractSimplex(simplex, centroid, phi);
119             functionCallsCount += calls;
120             isNegative = true;
121         }
122         return (simplex, functionCallsCount, isNegative);
123     }
124
125     private double[][] ShrinkSimplex(double[][] simplex)
126     {
127         double x = simplex[0][0];
128         double y = simplex[0][1];
129         double x2 = (x + simplex[1][0]) / 2;
130         double y2 = (y + simplex[1][1]) / 2;

```

```

131     double x3 = (x + simplex[2][0]) / 2;
132     double y3 = (y + simplex[2][1]) / 2;
133     double[][] newSimplex =
134         [
135             new double[] { x, y, simplex[0][2] },
136             new double[] { x2, y2, ObjectiveFunction(x2, y2) },
137             new double[] { x3, y3, ObjectiveFunction(x3, y3) }
138         ];
139     return newSimplex;
140 }
141
142 private double[] CalculateCentroid(double[][] simplex, double[] exclude)
143 {
144     int length = 2; // simplex[0].Length;
145     double[] centroid = new double[length + 1];
146     foreach (var point in simplex)
147     {
148         if (point != exclude)
149         {
150             for (int i = 0; i < length; i++)
151                 centroid[i] += point[i];
152         }
153     }
154
155     length = 2; //simplex.Length -1;
156     for (int i = 0; i < length; i++)
157         centroid[i] /= length;
158     return centroid;
159 }
160
161 private double[] Reflect(double[] centroid, double[] worst, double phi)
162 {
163     int length = worst.Length;
164     double[] reflected = new double[length];
165     for (int i = 0; i < length - 1; i++)
166         reflected[i] = worst[i] + (1 + phi) * (centroid[i] - worst[i]);
167     return reflected;
168 }
169
170 private double[][] GenerateSimplex(double x, double y, double edgeSize)
171 {
172     double x2 = x + edgeSize;
173     double y2 = y;
174     double x3 = x + edgeSize / 2;
175     double y3 = y + Math.Sqrt(Math.Pow(edgeSize, 2) - Math.Pow(edgeSize / 2, 2));
176
177     double[][] simplex =

```

```

178         [
179             new double[] { x, y, ObjectiveFunction(x, y) },
180             new double[] { x2, y2, ObjectiveFunction(x2, y2) },
181             new double[] { x3, y3, ObjectiveFunction(x3, y3) },
182         ];
183         return simplex;
184     }
185
186     private double[][] GenerateSimplex2(double x, double y, double edgeSize)
187     {
188         double beta1 = (Math.Sqrt(3) + 1) / (2 * Math.Sqrt(2)) * edgeSize;
189         double beta2 = (Math.Sqrt(3) - 1) / (2 * Math.Sqrt(2)) * edgeSize;
190         // ( $\sqrt{3} + 1$ ) /  $2\sqrt{2}$  ir ( $\sqrt{3} - 1$ ) /  $2\sqrt{2}$ 
191
192         double x2 = x + beta1;
193         double y2 = y + beta2;
194         double x3 = x + beta2;
195         double y3 = y + beta1;
196
197         double[][] simplex =
198         {
199             new double[] { x, y, ObjectiveFunction(x, y) },
200             new double[] { x2, y2, ObjectiveFunction(x2, y2) },
201             new double[] { x3, y3, ObjectiveFunction(x3, y3) },
202         };
203         return simplex;
204     }
205
206     private double getLongestEdgeLength(double[][] simplex)
207     {
208         double x = Math.Sqrt(Math.Pow(simplex[0][0] - simplex[1][0], 2) +
209             Math.Pow(simplex[0][1] - simplex[1][1], 2));
210         double y = Math.Sqrt(Math.Pow(simplex[0][0] - simplex[2][0], 2) +
211             Math.Pow(simplex[0][1] - simplex[2][1], 2));
212         double z = Math.Sqrt(Math.Pow(simplex[1][0] - simplex[2][0], 2) +
213             Math.Pow(simplex[1][1] - simplex[2][1], 2));
214
215         return Math.Max(x, Math.Max(y, z));
216     }
217 }
218 }

```

4.1. Deformuoto simplekso rezultatai

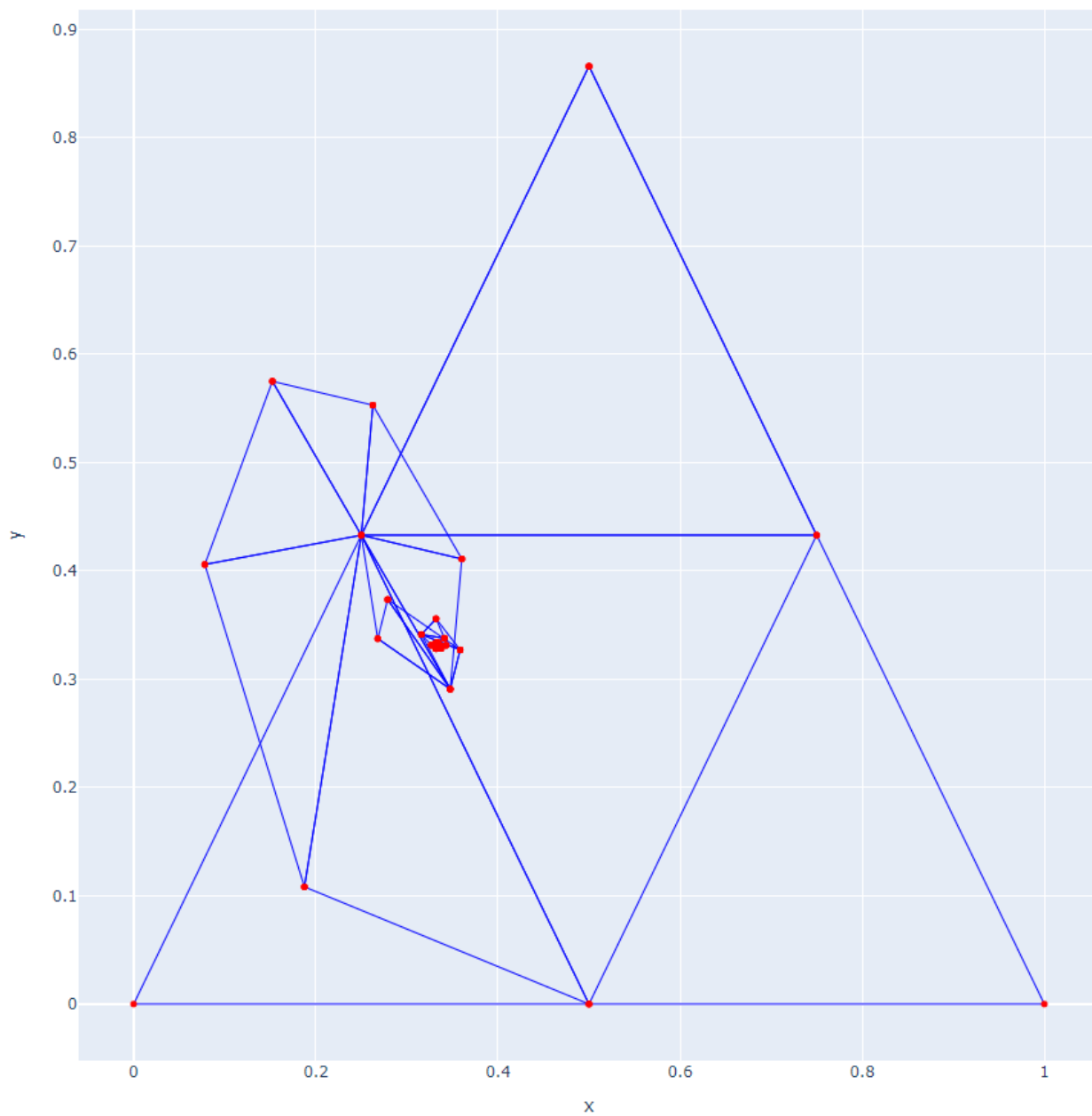
Lentelėse surašyti simplekso taškai per kiekvieną iteraciją. Naudojami sutartiniai taškų ženkliniai [4].

5. Deformuotas simpleksas. Pradinis taškas 0; 0)

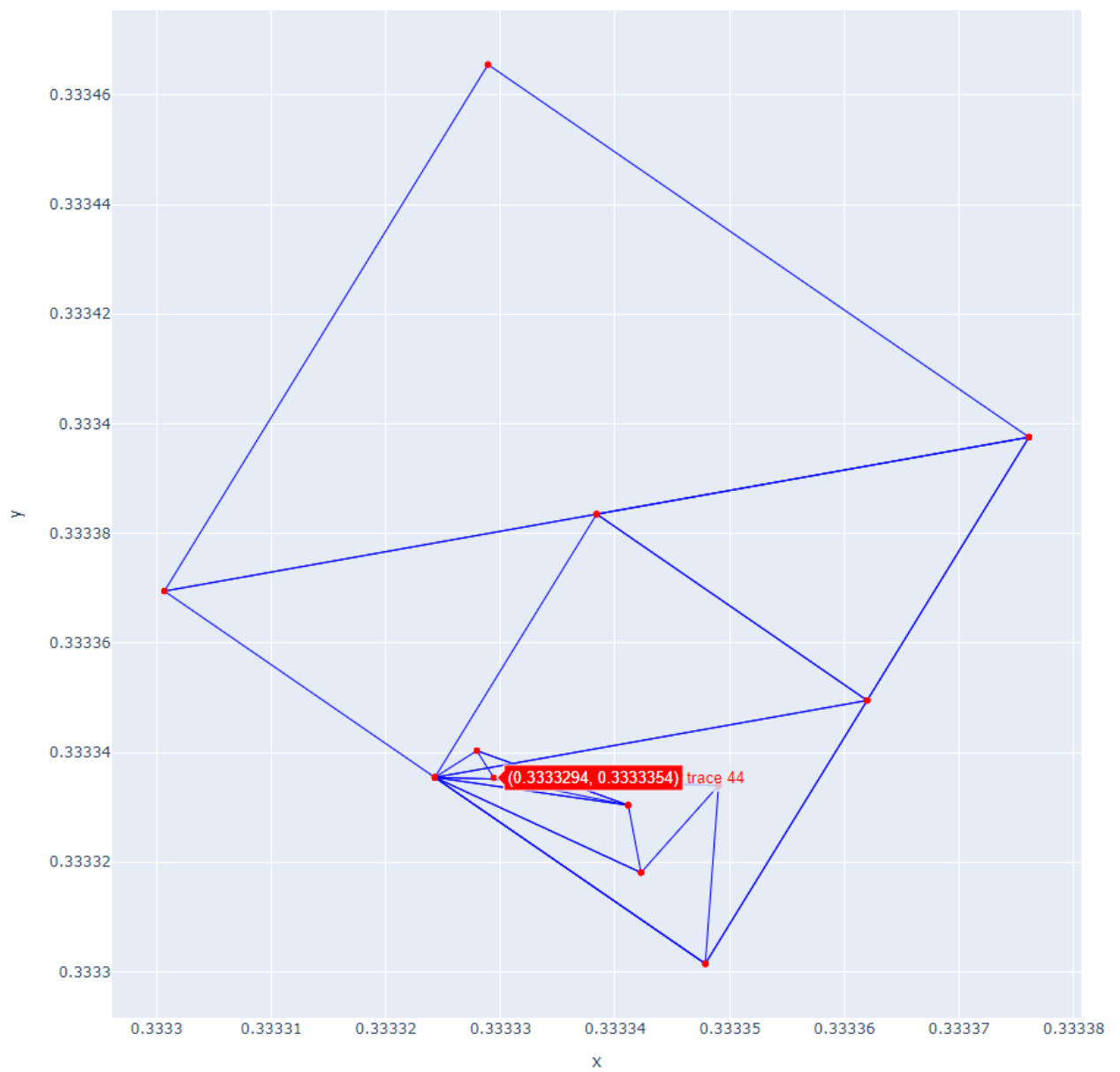
Tikslo funkcija kviesta 107 kartu, per 45 ciklus.

5 lentelė. Simplekso tarpinės taškų koordinatės su pradiniu tašku (0, 0)

X_w		X_v		X_b	
x_w	y_w	x_v	y_v	x_b	y_b
0.5	0.8660254037844386	1	0	0	0
0.5	0.8660254037844386	0.75	0.4330127018922193	0.25	0.4330127018922193
0.75	0.4330127018922193	0.5	0	0.25	0.4330127018922193
0.5	0	0.1875	0.1082531754730548	0.25	0.4330127018922193
0.1875	0.1082531754730548	0.078125	0.40594940802395557	0.25	0.4330127018922193
0.078125	0.40594940802395557	0.15234375	0.5750949947006037	0.25	0.4330127018922193
0.15234375	0.5750949947006037	0.2626953125	0.5531060684326393	0.25	0.4330127018922193
0.2626953125	0.5531060684326393	0.3603515625	0.41102377562425496	0.25	0.4330127018922193
0.3603515625	0.41102377562425496	0.25	0.4330127018922193	0.34765625	0.29093040908383494
0.25	0.4330127018922193	0.26806640625	0.33744544541991317	0.34765625	0.29093040908383494
0.26806640625	0.33744544541991317	0.2789306640625	0.3736003145720467	0.34765625	0.29093040908383494
0.2789306640625	0.3736003145720467	0.34765625	0.29093040908383494	0.3585205078125	0.32708527823596845
0.34765625	0.29093040908383494	0.3585205078125	0.32708527823596845	0.316009521484375	0.34130407911597416
0.3585205078125	0.32708527823596845	0.33206939697265625	0.3558268134720395	0.316009521484375	0.34130407911597416
0.33206939697265625	0.3558268134720395	0.316009521484375	0.34130407911597416	0.3412799835205078	0.3378253622649876
.	.	.	.	.	.
.	.	.	.	.	.
0.33334786828664686	0.3333014730244952	0.33334902853433424	0.3333339927520137	0.33332427934836373	0.333335468492659
0.33334902853433424	0.3333339927520137	0.3333422611139979	0.33331810182341576	0.33332427934836373	0.333335468492659
0.3333422611139979	0.33331810182341576	0.33332427934836373	0.333335468492659	0.3333411493827575	0.33333038895502554
0.33332427934836373	0.333335468492659	0.3333411493827575	0.33333038895502554	0.333327940991342	0.3333403421740555
0.3333411493827575	0.33333038895502554	0.333327940991342	0.3333403421740555	0.33332941226770674	0.33333541702859976



11 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš $(0, 0)$



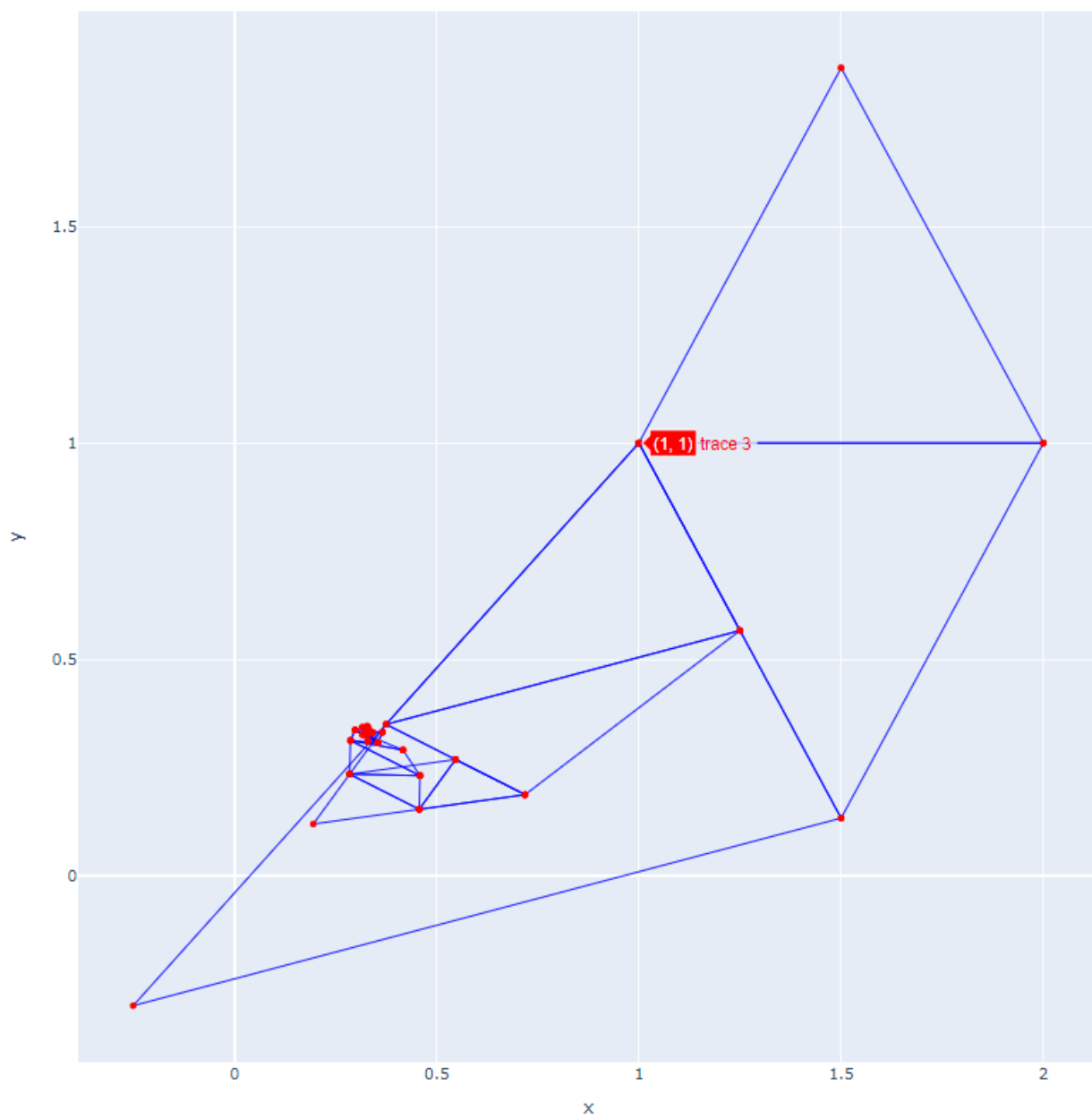
12 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš (0, 0), 10 paskutinių iteracijų

6. Deformuotas simpleksas. Pradinis taškas 1; 1)

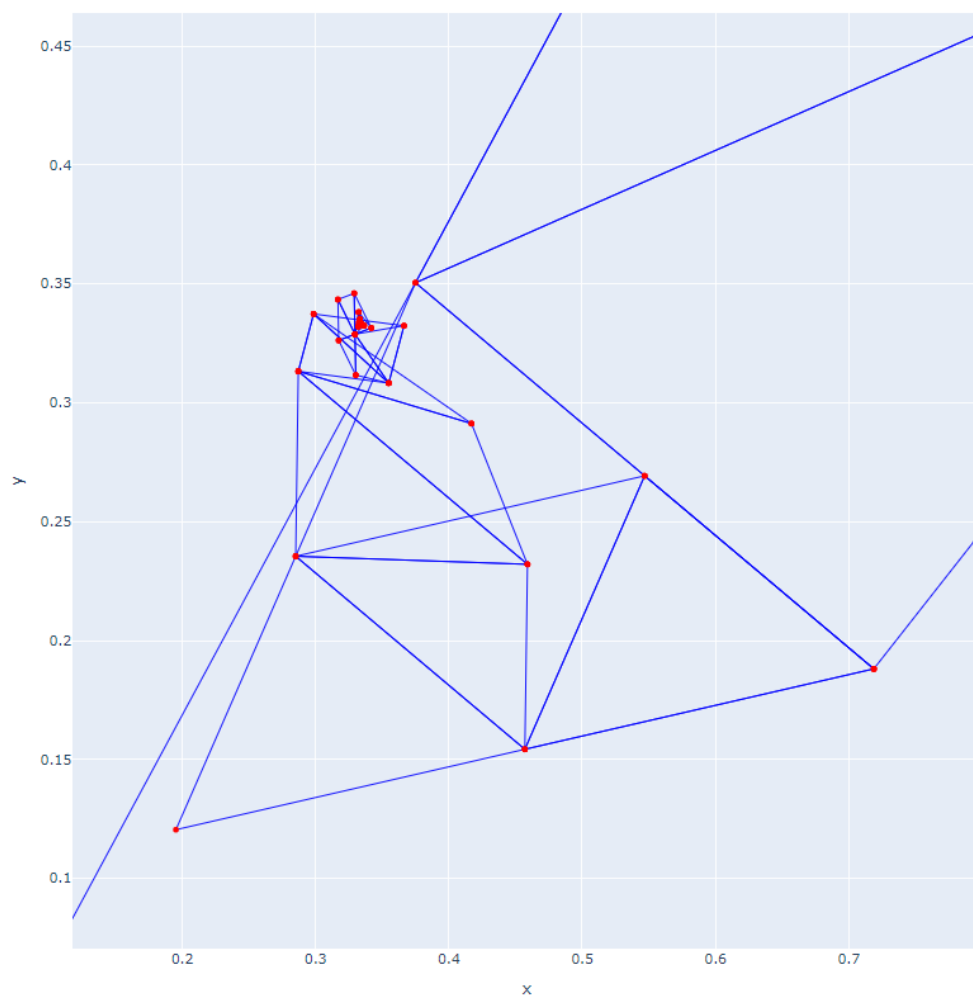
Tikslo funkcija kviesta 112 kartu, per 50 ciklus.

6 lentelė. Simplekso tarpinės taškų koordinatės su pradiniu tašku (1, 1)

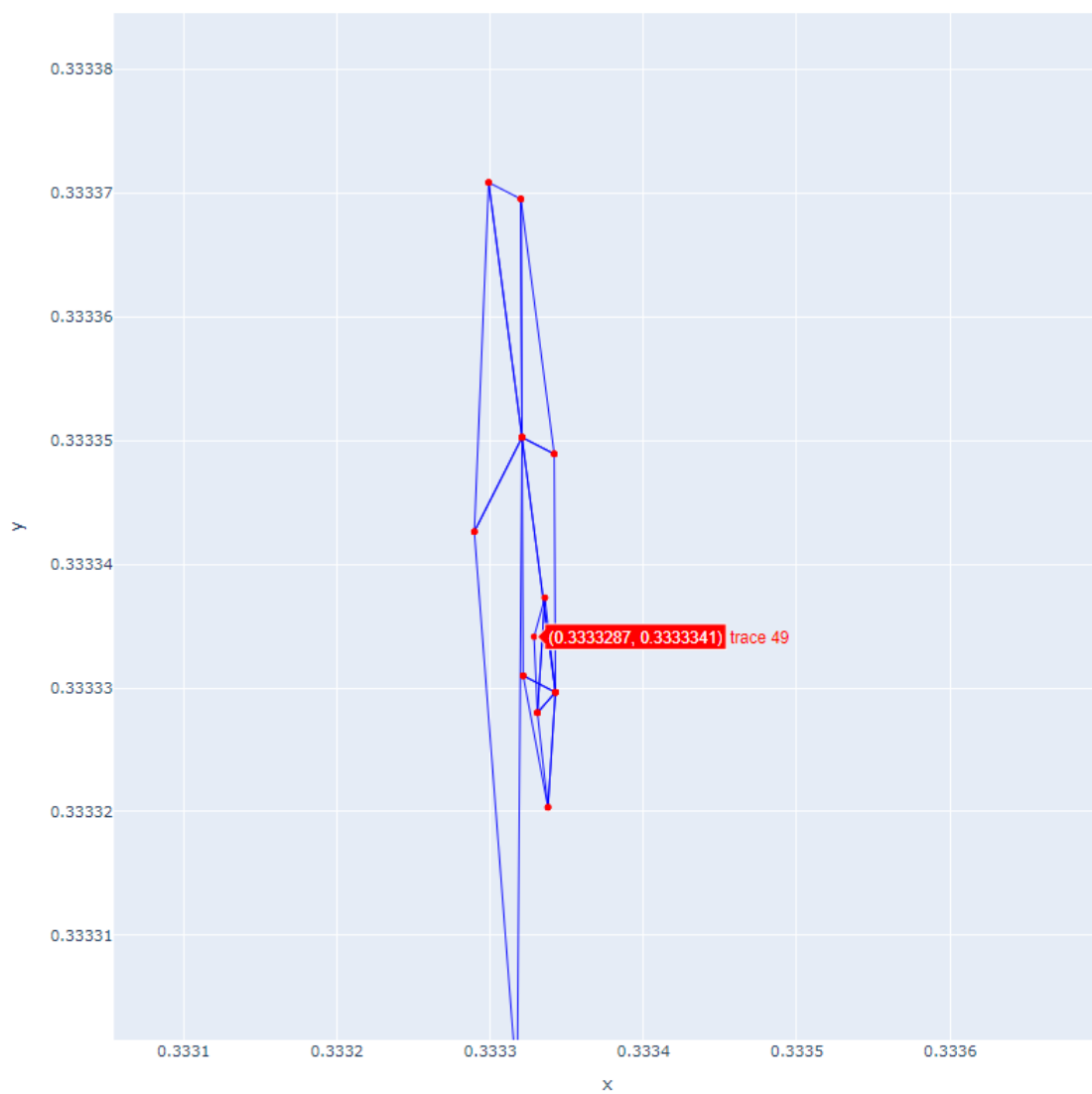
X_w		X_v		X_b	
x_w	y_w	x_v	y_v	x_b	y_b
1.5	1.8660254037844386	2	1	1	1
2	1	1	1	1.5	0.1339745962155614
1	1	1.5	0.1339745962155614	-0.25	-0.299038105676658
1	1	1.25	0.5669872981077807	0.375	0.350480947161671
1.25	0.5669872981077807	0.71875	0.18810118395208886	0.375	0.350480947161671
0.71875	0.18810118395208886	0.1953125	0.12044294928142951	0.375	0.350480947161671
0.71875	0.18810118395208886	0.546875	0.2692910655568799	0.45703125	0.15427206661675918
0.546875	0.2692910655568799	0.45703125	0.15427206661675918	0.28515625	0.23546194822155025
0.45703125	0.15427206661675918	0.28515625	0.23546194822155025	0.458984375	0.2320790364880173
0.28515625	0.23546194822155025	0.458984375	0.2320790364880173	0.287109375	0.3132689180928084
0.458984375	0.2320790364880173	0.4169921875	0.2912799918248442	0.287109375	0.3132689180928084
0.4169921875	0.2912799918248442	0.287109375	0.3132689180928084	0.298583984375	0.33737216419423083
0.287109375	0.3132689180928084	0.298583984375	0.33737216419423083	0.35491943359375	0.3083002664841819
0.298583984375	0.33737216419423083	0.36639404296875	0.33240351258560435	0.35491943359375	0.3083002664841819
0.36639404296875	0.33240351258560435	0.35491943359375	0.3083002664841819	0.329620361328125	0.328862026864562
0.35491943359375	0.3083002664841819	0.33020782470703125	0.31166996371875577	0.329620361328125	0.328862026864562
.	.	.	.	.	.
.	.	.	.	.	.
0.33332086834690244	0.3333502485260751	0.3333216324857864	0.33333098661208876	0.33334262187059505	0.3333296501790653
0.3333216324857864	0.33333098661208876	0.33333775659383486	0.333320353330328	0.33334262187059505	0.3333296501790653
0.33333775659383486	0.333320353330328	0.33334262187059505	0.3333296501790653	0.3333309108590007	0.3333279941833927
0.33334262187059505	0.3333296501790653	0.3333309108590007	0.3333279941833927	0.3333357761357609	0.33333729103213
0.3333309108590007	0.3333279941833927	0.3333357761357609	0.33333729103213	0.3333287043107736	0.3333341388221094



13 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš (1, 1)



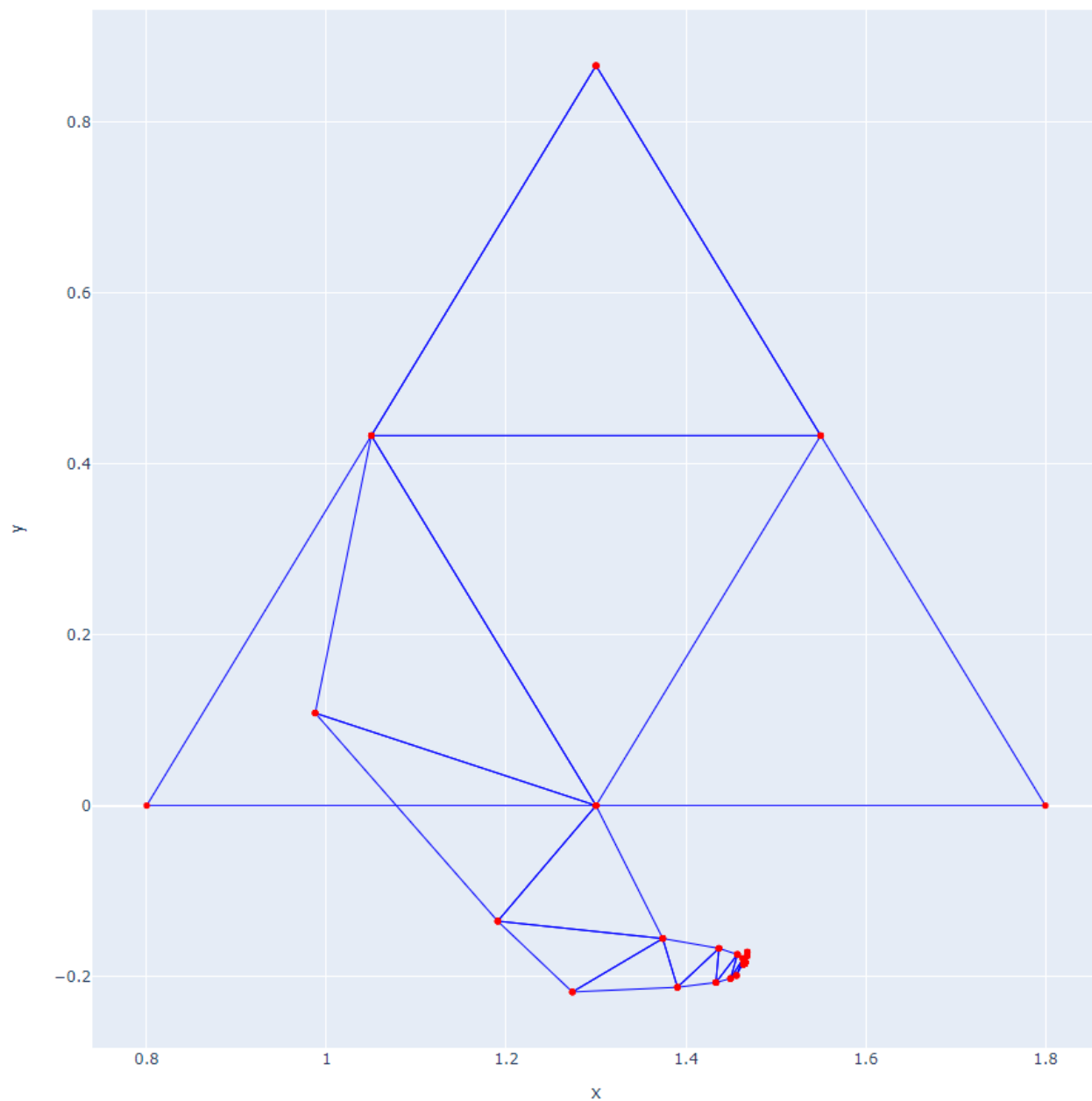
14 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš (1, 1) (priartintas)



15 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš (1, 1), 10 paskutinių iteracijų

7. Deformuotas simpleksas. Pradinis taškas (0,8; 0)

Tikslo funkcija kviesta 53 kartu, per 46 ciklus.



16 pav. Deformuoto simplekso kelionė link funkcijos minimumo iš (1, 1)

8. Visų metodų rezultatai

Paskutinės x ir y reikšmės priartėjo prie $\frac{1}{3}$, tai reiškia, jog priartėta prie funkcijos minimumo [1.3].

7 lentelė. Rezultatų lentelė

	Ciklai	Funkcijos kvietimai
Pradinis taškas (0; 0)		
Gradientinis nusileidimas	-	-
Greičiausias nusileidimas	-	-
Deformuotas simpleksas	107	45
Pradinis taškas (1; 1)		
Gradientinis nusileidimas	120	59
Greičiausias nusileidimas	34	29
Deformuotas simpleksas	112	50
Pradinis taškas (0,8; 0)		
Gradientinis nusileidimas	392	195
Greičiausias nusileidimas	178	65
Deformuotas simpleksas	53	46

9. Išvados

Geriausias metodas, iš tyrinėtų, būtų greičiausias nusileidimas. Šis metodas randa minimumą per, palyginus, labai mažą ciklų skaičių ir jis yra pakankamai lengvas implementuoti - parašyti kodą. Tačiau deformuoto simplekso metodas yra taip pat geras. Jo privalumas - galima rasti minimumą iš bet kokio taško, kai su greičiausiu nusileidimu nebuvo galima to padaryti su $(0, 0)$ tašku. Anaiptol, deformuotą simpleksą yra kurkas sudėtingiau implementuoti nei kitus metodus. Gradientinis nusileidimas pagal rezultatus prasčiausias metodas, nors lengviausiai implementuojamas, tačiau gradientinis nusileidimas tiesiog yra prastesnė greičiausio nusileidimo versija.